

SENAI - CRTI

Apostila

Consumo de APIs no Angular

Projeto guiado: ViaCep — passo a passo para o aluno replicar sozinho

Interação com APIs — Sprint 3

Sumário

- 1. Introdução e pré-requisitos
- 2. Conceitos básicos
- 3. Passo a passo: criando o projeto pelo terminal
- 4. Código-fonte completo, arquivo por arquivo
- 5. Executando e testando a aplicação
- 6. Testes automatizados
- 7. Build de produção

1. Introdução e pré-requisitos

Nesta apostila você vai construir, do zero, uma aplicação Angular chamada ViaCep. O usuário digita um CEP, a aplicação consulta a API pública ViaCEP e exibe rua, bairro, cidade e estado na tela, sem recarregar a página.

Siga os capítulos na ordem. Cada comando de terminal e cada arquivo de código aparecem completos, prontos para copiar.

Pré-requisitos

- Node.js instalado (versão LTS mais recente).
- Angular CLI instalado globalmente.
- Um editor de código (ex.: VS Code).

Para instalar o Angular CLI globalmente, caso ainda não tenha:

```
$ npm install -g @angular/cli
```

Para conferir se tudo está pronto, rode os três comandos abaixo. Eles devem responder com um número de versão, sem erro:

```
$ node --version  
$ npm --version  
$ ng version
```

2. Conceitos básicos

Antes de programar, é importante entender o papel de cada peça que vamos usar.

HttpClient

É o serviço oficial do Angular para fazer requisições HTTP (GET, POST, PUT, DELETE...). É ele quem efetivamente conversa com a API ViaCEP pela rede.

Service

Uma classe separada do componente, responsável apenas por buscar dados. Isolar essa lógica em um service evita duplicar código e deixa a aplicação mais fácil de manter e testar.

Model (Interface)

Uma interface TypeScript que descreve o formato dos dados que a API devolve (quais campos existem e de que tipo). Isso dá autocomplete e evita erros de digitação nos nomes dos campos.

Signals

Signals são variáveis reativas do Angular. Quando o valor de um signal muda (ex.: `signal.set(novoValor)`), a tela é atualizada automaticamente, sem precisar de comandos manuais de renderização.

Observable e subscribe

O HttpClient não devolve o dado na hora — ele devolve um Observable, que representa uma resposta que vai chegar no futuro. Para receber esse valor quando ele chegar, usamos `.subscribe(...)`.

Zoneless (Angular 21+)

Desde a versão 21, o Angular não depende mais da biblioteca Zone.js para saber quando atualizar a tela. Em vez disso, a atualização acontece diretamente quando um Signal muda de valor. Isso deixa a aplicação mais leve e rápida, sem exigir nenhuma configuração extra — já vem assim por padrão em projetos novos.

3. Passo a passo: criando o projeto pelo terminal

Abra o terminal na pasta onde deseja criar o projeto e siga os comandos na ordem.

3.1 Criar o projeto Angular

```
$ ng new ViaCep --style=css --routing=false --ssr=false --defaults
```

O que cada opção faz:

- `--style=css`: usa CSS puro para os estilos.
- `--routing=false`: a aplicação tem uma única tela, não precisamos de rotas.
- `--ssr=false`: aplicação apenas no navegador (client-side), sem Server-Side Rendering.
- `--defaults`: aceita as demais respostas padrão automaticamente, sem perguntas interativas.

Entre na pasta do projeto criado:

```
$ cd ViaCep
```

3.2 Gerar o model (interface) de CEP

```
$ ng generate interface models/cep
```

Esse comando cria o arquivo `src/app/models/cep.ts`, com uma interface `Cep` vazia. Vamos preencher os campos dela no próximo capítulo.

3.3 Gerar o service de CEP

```
$ ng generate service services/cep
```

Esse comando cria dois arquivos: `src/app/services/cep.ts` (o service) e `src/app/services/cep.spec.ts` (o arquivo de teste). O service já nasce decorado com `@Service()` — o Angular 22 já registra a classe automaticamente na aplicação, sem precisar escrever `providedIn: 'root'`.

3.4 Estrutura final de pastas

Depois de todos os comandos, e com os arquivos preenchidos (próximo capítulo), a estrutura deve ficar assim:

```
src/  
  main.ts  
  app/  
    app.config.ts  
    app.ts  
    app.html  
    app.css  
    app.spec.ts  
    models/  
      cep.ts  
    services/  
      cep.ts
```


4. Código-fonte completo, arquivo por arquivo

A seguir está o conteúdo final de cada arquivo. Abra cada um no editor e substitua o conteúdo gerado pelo Angular CLI pelo código abaixo.

4.1 src/main.ts

Ponto de entrada da aplicação. Inicializa (bootstrap) o componente principal App usando as configurações definidas em appConfig. Este arquivo é gerado pronto pelo Angular CLI — não precisa alterá-lo.

Arquivo: [src/main.ts](#)

```
import { bootstrapApplication } from "@angular/platform-browser";
import { appConfig } from "../app/app.config";
import { App } from "../app/app";

bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

4.2 src/app/app.config.ts

Configura os providers globais da aplicação. Aqui é onde habilitamos o HttpClient com provideHttpClient() — sem essa linha, os services não conseguem fazer requisições HTTP.

Arquivo: [src/app/app.config.ts](#)

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from "@angular/core";
import { provideHttpClient } from "@angular/common/http";
export const appConfig: ApplicationConfig = {
  providers: [provideBrowserGlobalErrorListeners(), provideHttpClient()],
};
```

4.3 src/app/models/cep.ts

Interface que descreve o formato do JSON devolvido pela API ViaCEP. O campo erro é opcional (?) porque só aparece na resposta quando o CEP não é encontrado.

Arquivo: [src/app/models/cep.ts](#)

```
export interface Cep {
  cep: string;
  logradouro: string;
  complemento: string;
  bairro: string;
  localidade: string;
  uf: string;
  estado: string;
  erro?: boolean;
}
```

4.4 src/app/services/cep.ts

O service concentra toda a comunicação com a API. O método buscarCep recebe o CEP digitado, monta a URL da ViaCEP com template literal e devolve um Observable tipado com <Cep> — assim o TypeScript já sabe quais campos a resposta vai ter.

Arquivo: [src/app/services/cep.ts](#)

```
import { inject } from "@angular/core";
import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Cep } from "../models/cep";
@Injectable({
  providedIn: 'root'
})
export class CepService {
  private http = inject(HttpClient);
  buscarCep(cep: string) {
    return this.http.get<Cep>(`https://viacep.com.br/ws/${cep}/json/`);
  }
}
```

Observação: a linha da URL usa concatenação de string acima apenas por clareza de impressão; no editor, prefira o template literal do TypeScript, mostrado abaixo:

```
return this.http.get<Cep>(`https://viacep.com.br/ws/${cep}/json/`);
```

4.5 src/app/app.ts

O componente principal. Guarda o estado da tela em três signals (endereço, loading, erro) e usa um FormControl para validar que o CEP tem exatamente 8 dígitos numéricos antes de chamar o service.

Arquivo: [src/app/app.ts](#)

```
import { Component, inject, signal } from "@angular/core";
import { FormControl, ReactiveFormsModule, Validators } from "@angular/forms";
import { CepService } from "../services/cep";
import { Cep } from "../models/cep";

@Component({
  selector: "app-root",
  imports: [ReactiveFormsModule],
  templateUrl: "./app.html",
  styleUrls: ["./app.css"],
})
export class App {
  private cepService = inject(CepService);

  cepControl = new FormControl("", [Validators.required, Validators.pattern(/^\d{8}$/)]);

  endereco = signal<Cep | null>(null);
  loading = signal(false);
  erro = signal("");

  buscar() {
    if (this.cepControl.invalid) {
      this.erro.set("Digite um CEP válido com 8 números.");
      return;
    }

    this.loading.set(true);
    this.erro.set("");
    this.endereco.set(null);

    const cep = this.cepControl.value!;

    this.cepService.buscarCep(cep).subscribe({
```



```

    next: (resposta) => {
      this.loading.set(false);

      if (resposta.erro) {
        this.erro.set("CEP não encontrado.");
        return;
      }

      this.endereco.set(resposta);
    },
    error: () => {
      this.loading.set(false);
      this.erro.set("Erro ao consultar API.");
    },
  });
}
}

```

4.6 src/app/app.html

Template da tela: campo de texto ligado ao cepControl, botão de busca, e três blocos condicionais (@if) que mostram, respectivamente, o carregamento, a mensagem de erro e o card com o endereço encontrado.

Arquivo: [src/app/app.html](#)

```

<div class="container">
  <h1>Consulta CEP</h1>

  <div class="busca">
    <input
      type="text"
      placeholder="Digite o CEP (somente números)"
      [formControl]="cepControl"
    />
    <button (click)="buscar()">Buscar</button>
  </div>

  @if (loading()) {
    <p class="loading">Consultando...</p>
  }

  @if (erro()) {
    <p class="erro">{{ erro() }}</p>
  }

  @if (endereco(); as cep) {
    <div class="card">
      <h2>Endereço encontrado</h2>
      <p><strong>CEP:</strong> {{ cep.cep }}</p>
      <p><strong>Rua:</strong> {{ cep.logradouro }}</p>
      <p><strong>Bairro:</strong> {{ cep.bairro }}</p>
      <p><strong>Cidade:</strong> {{ cep.localidade }}</p>
      <p><strong>UF:</strong> {{ cep.uf }}</p>
    </div>
  }
</div>

```

4.7 src/app/app.css

Estilos da tela: layout centralizado, campo de busca, botão azul e o card branco que exibe o endereço.

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
  display: block;
  background: #f5f7fb;
  min-height: 100vh;
}

.container {
  width: 100%;
  max-width: 700px;
  margin: 60px auto;
  padding: 30px;
}

h1 {
  text-align: center;
  margin-bottom: 30px;
  color: #222;
}

.busca {
  display: flex;
  gap: 12px;
  margin-bottom: 25px;
}

input {
  flex: 1;
  padding: 14px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 16px;
}

button {
  border: none;
  background: #2563eb;
  color: white;
  padding: 14px 24px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
}

button:hover {
  background: #1d4ed8;
}

.card {
  background: white;
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
}

.card h2 {
  margin-bottom: 15px;
}
```

```
    color: #111;
}

.card p {
  margin: 8px 0;
  color: #444;
}

.loading {
  color: #2563eb;
  margin-bottom: 20px;
  font-weight: bold;
}

.erro {
  color: #dc2626;
  margin-bottom: 20px;
  font-weight: bold;
}
```

5. Executando e testando a aplicação

Com todos os arquivos salvos, suba o servidor de desenvolvimento:

```
$ npm start
```

Ou, de forma equivalente:

```
$ ng serve
```

Abra o navegador em <http://localhost:4200> e teste os três cenários abaixo.

Roteiro de teste

- CEP válido, ex.: 20040002 → deve exibir o card com rua, bairro, cidade e UF.
- CEP inexistente, ex.: 00000000 → deve exibir a mensagem "CEP não encontrado."
- CEP incompleto ou com letras, ex.: 123 → deve exibir "Digite um CEP válido com 8 números." sem chegar a chamar a API.

6. Testes automatizados

O Angular 22 usa o Vitest como executor de testes por padrão. Os arquivos `.spec.ts` abaixo já verificam se o componente e o service são criados corretamente.

6.1 src/app/app.spec.ts

Arquivo: [src/app/app.spec.ts](#)

```
import { TestBed } from "@angular/core/testing";
import { provideHttpClient } from "@angular/common/http";
import { App } from "./app";

describe("App", () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
      providers: [provideHttpClient()],
    }).compileComponents();
  });

  it("should create the app", () => {
    const fixture = TestBed.createComponent(App);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });

  it("should render title", async () => {
    const fixture = TestBed.createComponent(App);
    await fixture.whenStable();
    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector("h1")?.textContent).toContain("Consulta CEP");
  });
});
```

6.2 src/app/services/cep.spec.ts

Arquivo: [src/app/services/cep.spec.ts](#)

```
import { TestBed } from "@angular/core/testing";
import { provideHttpClient } from "@angular/common/http";
import { CepService } from "./cep";

describe("CepService", () => {
  let service: CepService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient()],
    });
    service = TestBed.inject(CepService);
  });

  it("should be created", () => {
    expect(service).toBeTruthy();
  });
});
```

6.3 Rodando os testes

```
$ ng test --watch=false
```

O resultado esperado é a mensagem de todos os testes passando ("Tests ... passed").

7. Build de produção

Quando o projeto estiver pronto para publicar, gere a versão otimizada com:

```
$ ng build
```

Os arquivos finais são gerados na pasta dist/ViaCep, prontos para hospedar em qualquer servidor de arquivos estáticos.